



DECODELABS INTERNSHIP PROGRAM

Robotics and Automation Project Report



JULY 6, 2026

NAME: HIMANSHA PATABADIGE

GITHUB: <https://github.com/HimanshaPatabdige>

PROJECT 1 – ROBOTIC ARM KINEMATICS AND PATH PLANNING

INTRODUCTION

Robotic arms are widely used in industrial automation for performing repetitive and precise tasks. This project demonstrates the basic concepts of robotic arm kinematics and path planning using Python.

OBJECTIVES

- Calculate robotic arm joint angles.
- Generate a path between points.
- Visualize the movement path.

SOFTWARE USED

- Python
- PyCharm
- Git
- GitHub
- Matplotlib

METHODOLOGY

Inverse Kinematics

- The target coordinates were provided and the joint angle was calculated using trigonometric functions.

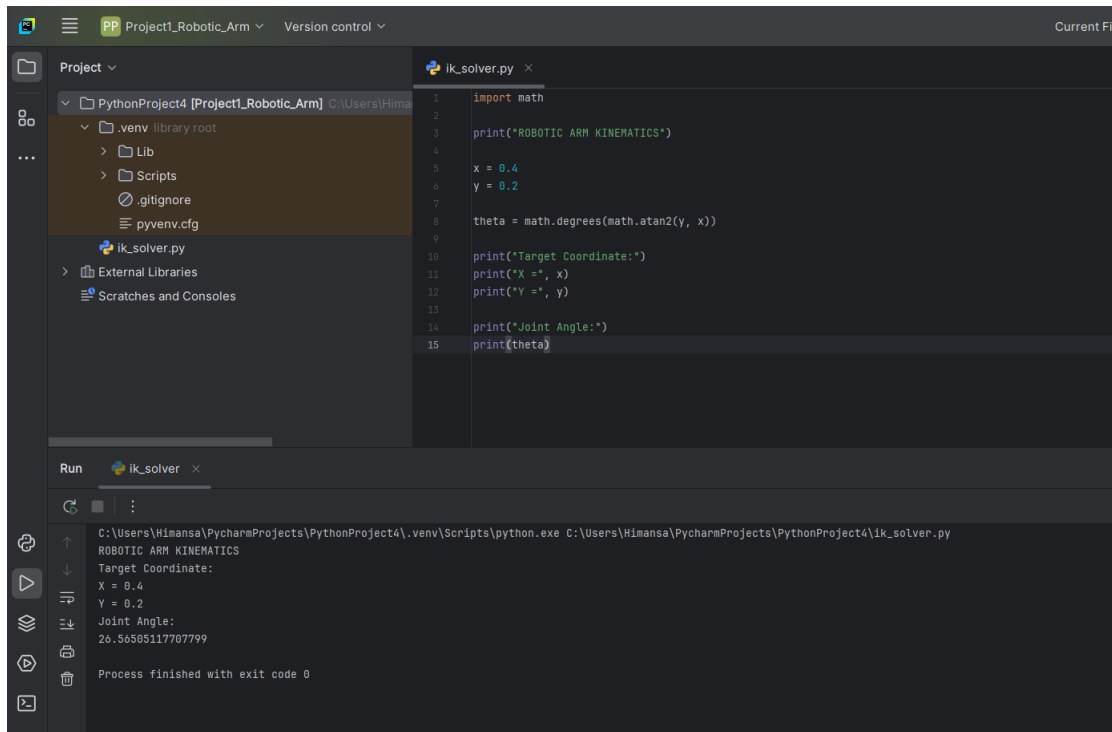
Path Planning

- A sequence of waypoints was created to define the robot's movement.

Visualization

- The generated path was plotted using Matplotlib.

RESULTS



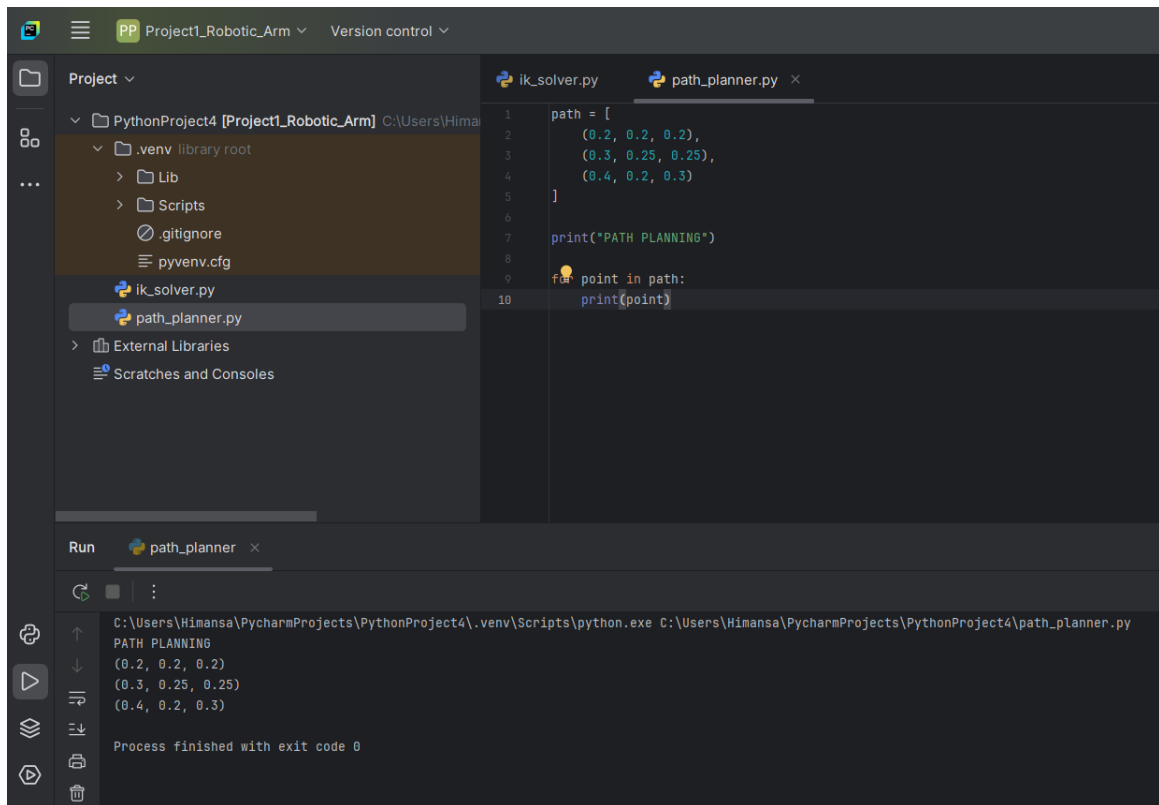
The screenshot displays the PyCharm IDE interface for a project named 'Project1_Robotic_Arm'. The left sidebar shows the project structure with files like 'ik_solver.py' and 'pyenv.cfg'. The main editor window shows the code for 'ik_solver.py'.

```
1 import math
2
3 print("ROBOTIC ARM KINEMATICS")
4
5 x = 0.4
6 y = 0.2
7
8 theta = math.degrees(math.atan2(y, x))
9
10 print("Target Coordinate:")
11 print("X =", x)
12 print("Y =", y)
13
14 print("Joint Angle:")
15 print(theta)
```

The bottom panel shows the output of the program:

```
ROBOTIC ARM KINEMATICS
Target Coordinate:
X = 0.4
Y = 0.2
Joint Angle:
26.56505117707799
Process finished with exit code 0
```

Figure 1: Inverse Kinematics Solver Implementation using Python



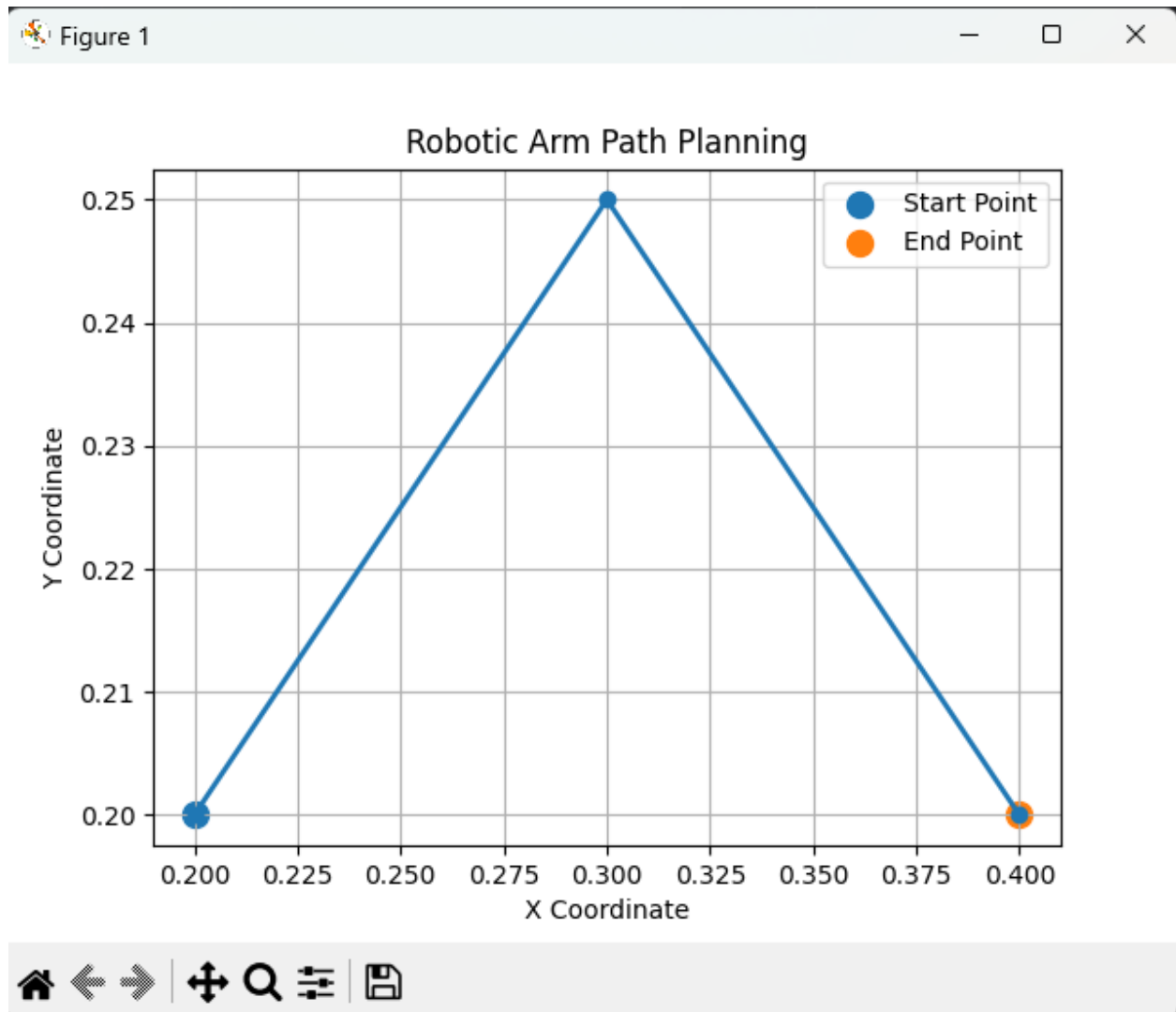
The screenshot displays the PyCharm IDE interface for the same project. The left sidebar shows the project structure with files like 'ik_solver.py' and 'path_planner.py'. The main editor window shows the code for 'path_planner.py'.

```
1 path = [
2     (0.2, 0.2, 0.2),
3     (0.3, 0.25, 0.25),
4     (0.4, 0.2, 0.3)
5 ]
6
7 print("PATH PLANNING")
8
9 for point in path:
10     print(point)
```

The bottom panel shows the output of the program:

```
PATH PLANNING
(0.2, 0.2, 0.2)
(0.3, 0.25, 0.25)
(0.4, 0.2, 0.3)
Process finished with exit code 0
```

Figure 2: Autonomous Path Planning Module for Robot Navigation



- **Figure 3: Graphical Visualization of Planned Robot Trajectory**

CONCLUSION

The project successfully demonstrated robotic arm kinematics and path planning concepts. Joint angles were calculated, movement paths were generated, and the trajectory was visualized using Python.